

INDEX

S.N.	Table of Contents (Program Description / Topic)	Signature
1	WAP to calculate the sum of two numbers using a class and constructor.	
2	WAP to reverse an array and display both the original and reversed arrays.	
3	WAP to store and display student names using a class and indexer.	
4	WAP to create a student class with automatic properties and display student details.	
5	WAP to demonstrate runtime polymorphism using virtual and override methods.	
6	WAP program to create a Calculator class with methods to perform addition, subtraction, multiplication, and division of two numbers.	
7	WAP to create a Rectangle class that implements two interfaces for Length and Width and calculates the area.	
8	WAP to implement Class and Methods	
9	WAP to Implement of Array	
10	Implementation of If and Switch Statements	
11	Implementation of For and Foreach Loop	
12	Implementation of While and Do-While Loop	

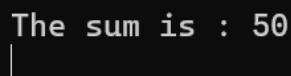
13	Implementation of Constructors	
14	Implementation of Inheritance	
15	Implementation of Abstract Class and Method	
16	Implementation of Method Overloading	
17	Implementation of Interface	
18	Implementation of Generics	
19	Implementation of Delegates	
20	Implementation of Lambda Expression	
21	Implementation of Exception Handling	
22	Implementation of LINQ	

1. Write a C# program to calculate the sum of two numbers using a class and constructor.

```
using System;

namespace AddTwoDigitsConstructor
{
    class Addition
    {
        int x, y;
        public Addition(int a, int b)
        {
            this.x = a;
            this.y = b;
        }
        public int Sum()
        {
            return x + y;
        }
    }
    class Program
    {
        static void Main(string[] args)
        {
            Addition obj = new Addition(30, 20);
            Console.WriteLine($"The sum is : {obj.Sum()}");
            Console.ReadLine();
        }
    }
}
```

OUTPUT

A screenshot of a console window with a black background. The text "The sum is : 50" is displayed in white. A white cursor line is visible on the line below the output text.

The sum is : 50

2. Write a C# program to reverse an array and display both the original and reversed arrays.

```
using System;

namespace LAB
{
    internal class Program
    {
        static void Main(string[] args)
        {
            // Original array
            int[] arr = { 14, 22, 13, 54, 5 };

            Console.WriteLine("Original Array:");
            foreach (int item in arr)
            {
                Console.Write(item + " ");
            }

            // Reverse the array
            Array.Reverse(arr);

            Console.WriteLine("\n\nReversed Array:");
            foreach (int item in arr)
            {
                Console.Write(item + " ");
            }

            Console.ReadLine(); // keep console open
        }
    }
}
```

OUTPUT

```
Original Array:
14 22 13 54 5

Reversed Array:
5 54 13 22 14
```

3. Write a C# program to store and display student names using a class and indexer.

```
using System;

namespace LAB
{
    class StudentNames
    {
        private string[] names = new string[5]; // Fixed size array

        // Indexer
        public string this[int index]
        {
            get { return names[index]; }
            set { names[index] = value; }
        }
    }

    internal class Program
    {
        static void Main(string[] args)
        {
            StudentNames students = new StudentNames();

            // Store names using indexer
            students[0] = "Sandesh";
            students[1] = "Ashesh";
            students[2] = "Prakash";
            students[3] = "Pradip";
            students[4] = "Advik";

            Console.WriteLine("--- Student Names ---");
            for (int i = 0; i < 5; i++)
            {
                Console.WriteLine($"Student {i + 1}: {students[i]}");
            }
        }
    }
}
```

```

        Console.ReadLine(); // Keep console open
    }
}
}

```

OUTPUT

```

--- Student Names ---
Student 1: Sandesh
Student 2: Ashesh
Student 3: Prakash
Student 4: Pradip
Student 5: Advik

```

- 4. Write a C# program to create a student class with automatic properties and display student details**

```

using System;

namespace StudentApp
{
    class Student
    {
        // Automatic properties
        public int StudentId { get; set; }
        public string Name { get; set; }
    }

    class Program
    {
        static void Main(string[] args)
        {
            // Create and initialize student object
            Student s1 = new Student
            {
                StudentId = 10,
                Name = "Ayush"
            };
        }
    }
}

```

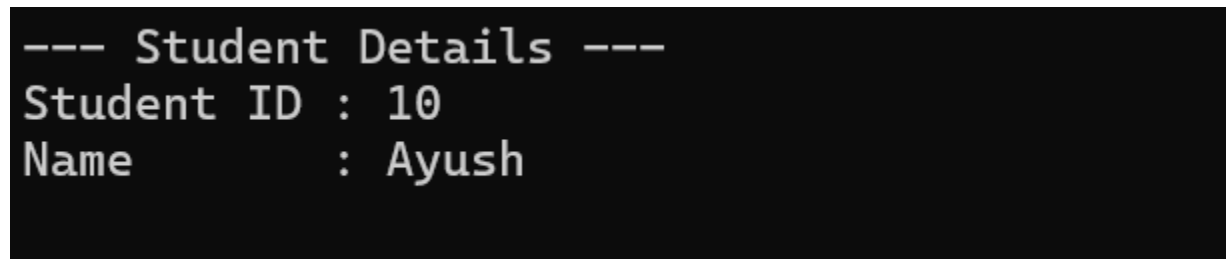
```

        // Display student details
        Console.WriteLine("--- Student Details ---");
        Console.WriteLine("Student ID : " + s1.StudentId);
        Console.WriteLine("Name      : " + s1.Name);

        Console.ReadLine();
    }
}

```

OUTPUT



```

--- Student Details ---
Student ID : 10
Name      : Ayush

```

- Write a C# program to demonstrate runtime polymorphism using virtual and override methods.**

```

using System;

namespace LAB
{
    // Base class
    class Animal
    {
        // Virtual method
        public virtual void MakeSound()
        {
            Console.WriteLine("Animal makes a sound");
        }
    }
}

```

```

// Derived class 1
class Dog : Animal
{
    // Override the virtual method
    public override void MakeSound()
    {
        Console.WriteLine("Dog barks");
    }
}

// Derived class 2
class Cat : Animal
{
    // Override the virtual method
    public override void MakeSound()
    {
        Console.WriteLine("Cat meows");
    }
}

internal class Program
{
    static void Main(string[] args)
    {
        Animal myAnimal = new Animal();
        Animal myDog = new Dog();
        Animal myCat = new Cat();
    }
}

```



```
Console.WriteLine("--- Polymorphism using Virtual Methods ---");  
myAnimal.MakeSound(); // Calls base class method  
myDog.MakeSound();    // Calls derived class method  
myCat.MakeSound();    // Calls derived class method  
  
Console.ReadLine();  
}  
}  
}
```

OUTPUT

```
--- Polymorphism using Virtual Methods ---  
Animal makes a sound  
Dog barks  
Cat meows  
|
```

6. Write a C# program to create a Calculator class with methods to perform addition, subtraction, multiplication, and division of two numbers.

```
using System;

namespace LAB
{
    // Calculator class
    class Calculator
    {
        public int Add(int a, int b) => a + b;

        public int Subtract(int a, int b) => a - b;

        public int Multiply(int a, int b) => a * b;

        public double Divide(double a, double b)
        {
            if (b == 0)
            {
                Console.WriteLine("Division by zero is not allowed!");
                return 0;
            }
            return a / b;
        }
    }

    internal class Program
    {
        static void Main(string[] args)
        {
            Calculator calc = new Calculator();

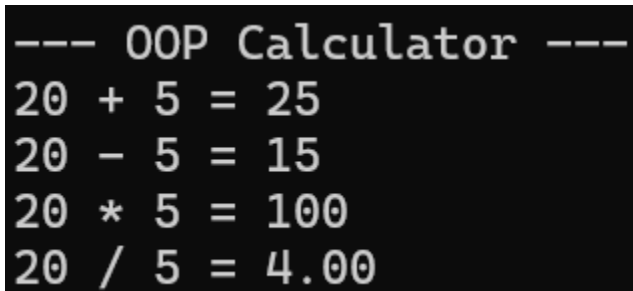
            int num1 = 20;
            int num2 = 5;

            Console.WriteLine("--- OOP Calculator ---");
        }
    }
}
```

```
        Console.WriteLine($"{num1} + {num2} = {calc.Add(num1, num2)}");
        Console.WriteLine($"{num1} - {num2} = {calc.Subtract(num1, num2)}");
        Console.WriteLine($"{num1} * {num2} = {calc.Multiply(num1,
num2)}");
        Console.WriteLine($"{num1} / {num2} = {calc.Divide(num1,
num2):F2}");
        // formatted to 2 decimals

        Console.ReadLine();
    }
}
```

OUTPUT

A screenshot of a terminal window with a black background and white text. The text shows the output of a program: a title line followed by four arithmetic operations. The title is "--- OOP Calculator ---". The operations are "20 + 5 = 25", "20 - 5 = 15", "20 * 5 = 100", and "20 / 5 = 4.00".

```
--- OOP Calculator ---
20 + 5 = 25
20 - 5 = 15
20 * 5 = 100
20 / 5 = 4.00
```

7. **Write a C# program to create a Rectangle class that implements two interfaces for Length and Width and calculates the area.**

```
using System;

namespace LAB
{
    // First interface for length
    interface ILength
    {
        double Length { get; set; }
    }

    // Second interface for width
    interface IWidth
    {
        double Width { get; set; }
    }

    // Class that inherits both interfaces
    class Rectangle : ILength, IWidth
    {
        public double Length { get; set; }
        public double Width { get; set; }

        public double GetArea()
        {
            return Length * Width;
        }
    }

    internal class Program
    {
        static void Main(string[] args)
        {
            Rectangle rect = new Rectangle
            {
```

```
        Length = 5,  
        Width = 10  
    };  
  
    Console.WriteLine($"Length: {rect.Length}");  
    Console.WriteLine($"Width: {rect.Width}");  
    Console.WriteLine($"Area of Rectangle: {rect.GetArea()}");  
  
    Console.ReadLine();  
}  
}  
}
```

OUTPUT

```
Length: 5  
Width: 10  
Area of Rectangle: 50
```

8. WAP to implement Class and Methods

Source Code:

```
using System;

namespace Project1
{
    internal class Class1
    {
        public class Calculation
        {
            //non parameterized method
            public void Add()
            {
                Console.WriteLine("This is non-parameterized
method");
            }

            //parameterized method
            public void Add(int x, int y)
            {
                Console.WriteLine("This is parameterized
method.\nSum:"+(x+y));
            }
        }

        static void Main(string[] args)
        {
            Calculation cal = new Calculation();
            cal.Add();
            cal.Add(10, 20);
            Console.ReadLine();
        }
    }
}
```

Output:

```
This is non-parameterized method  
This is parameterized method.  
Sum:30  
|
```

Conclusion:

From this lab, we learned to implement class and methods. Hence, a class Calculation is created and parameterized and non-parameterized methods are created and output is displayed.

9. WAP to Implement of Array

Source Code:

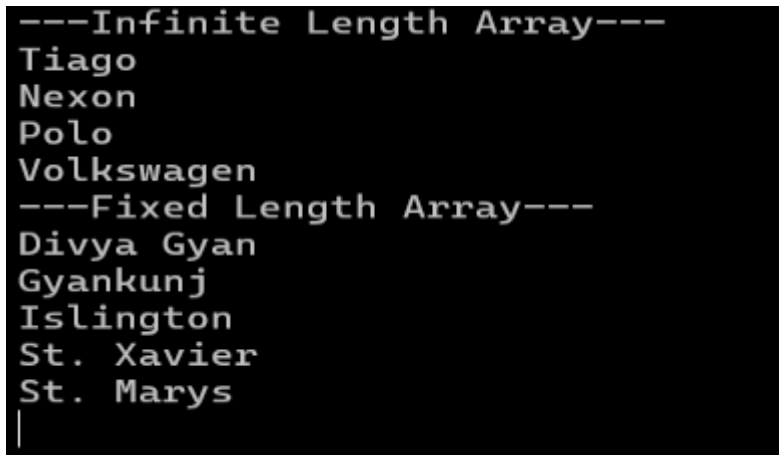
```
using System;  
namespace Chapter_Two_Arrays  
{  
    internal class Program  
    {  
        static void Main(string[] args)  
        {  
            //Infinite length Array  
            string[] cars;  
            cars = new string[] { "Tiago", "Nexon", "Polo",  
"Volkswagen" };  
  
            //Fixed length Array  
            string[] schools = new string[5];  
            schools[0] = "Divya Gyan";  
            schools[1] = "Gyankunj";  
            schools[2] = "Islington";  
            schools[3] = "St. Xavier";  
        }  
    }  
}
```

```

        schools[4] = "St. Marys";
        Console.WriteLine("---Infinite Length Array---");
        foreach (string s in cars) {
            Console.WriteLine(s);
        }
        Console.WriteLine("---Fixed Length Array---");
        foreach (string s in schools) {
            Console.WriteLine(s);
        }
        Console.ReadLine();
    }
}
}

```

Output:



```

---Infinite Length Array---
Tiago
Nexon
Polo
Volkswagen
---Fixed Length Array---
Divya Gyan
Gyankunj
Islington
St. Xavier
St. Marys
|

```

Conclusion:

From this lab, we learned to implement class and methods. Hence, a class Calculation is created and parameterized and non-parameterized methods are created and output is displayed.

10. Implementation of If and Switch Statements

Objective:

- To implement If and Switch statements

Source Code of If Statements:

```
using System;

namespace Chapter_Two_If_Statement
{
    internal class Program
    {
        static void Main(string[] args)
        {
            int number = 15;

            if (number > 20)
            {
                Console.WriteLine("The number is greater than 20.");
            }
            else if (number > 10)
            {
                Console.WriteLine("The number is greater than 10 but
less than or equal to 20.");
            }
            else
            {
                Console.WriteLine("The number is 10 or less.");
            }
            Console.ReadLine();
        }
    }
}
```

Output:

```
The number is greater than 10 but less than or equal to 20.
|
```

Source Code of Switch Statements:

```
using System;
namespace Chapter_Two_If_Statement
{
    internalclass Program
    {
        static void Main(string[] args)
        {
            int iNo;

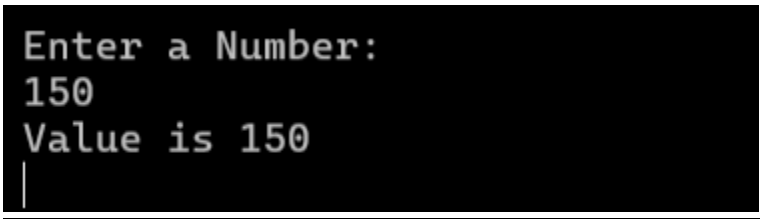
            Console.WriteLine("Enter a Number: ");
            iNo = Convert.ToInt32(Console.ReadLine());
            switch (iNo) {
                case 100:
                    Console.WriteLine("Value is 100");
                    break;

                case 50:
                    Console.WriteLine("Value is 50");
                    break;

                case 25:
                    Console.WriteLine("Value is 25");
                    break;

                default:
                    Console.WriteLine("Value is "+iNo);
                    break;
            }
            Console.ReadLine();
        }
    }
}
```

Output:

A screenshot of a terminal window with a black background and white text. It shows the output of the program: 'Enter a Number:', followed by the user input '150', and then the program output 'Value is 150'. A vertical cursor is visible at the end of the last line.

```
Enter a Number:
150
Value is 150
|
```

11. Implementation of For and Foreach Loop

Objective:

- To implement for and foreach loops

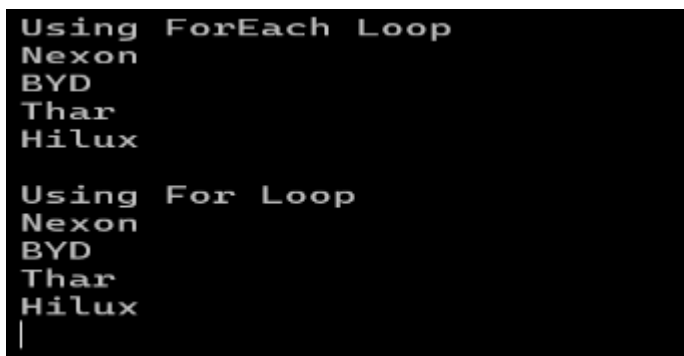
Source Code:

```
using System;
namespace Chapter_Two_For_Each_Loop
{
    internal class Program
    {
        static void Main(string[] args)
        {
            string[] strMotors = { "Nexon", "BYD", "Thar",
"Hilux" };
            Console.WriteLine("Using ForEach Loop");
            foreach (string s in strMotors)
                Console.WriteLine(s);

            Console.WriteLine("\nUsing For Loop");
            for (int i = 0; i < strMotors.Length; i++)
                Console.WriteLine(strMotors[i]);

            Console.ReadLine();
        }
    }
}
```

Output:



```
Using ForEach Loop
Nexon
BYD
Thar
Hilux

Using For Loop
Nexon
BYD
Thar
Hilux
|
```

Conclusion:

From this lab, we learned to implement for and foreach loop. Hence, elements of an array are displayed using for and foreach loop.

12. Implementation of While and Do-While Loop

Objective:

- To implement for and foreach loops

Source Code:

```
using System;
namespace Chapter_Two_Do_While_Loop
{
    internal class Program
    {
        static void Main(string[] args)
        {
            int iNo, iNo1;
            Console.WriteLine("Enter a number: ");
            iNo = Convert.ToInt32(Console.ReadLine());
            iNo1 = iNo;
            Console.WriteLine("---Using While Loop---");
            while (iNo > 0)
            {
                Console.WriteLine(iNo);
                iNo--;
            }
            Console.WriteLine("\n---Using Do-While Loop---");
            do
            {
                Console.WriteLine(iNo1);
                iNo1--;
            } while (iNo1 > 0);

            Console.ReadLine();
        }
    }
}
```

Output:

```
Enter a number:
5
---Using While Loop---
5
4
3
2
1

---Using Do-While Loop---
5
4
3
2
1
|
```

Conclusion:

From this lab, we learned to implement while and do-while loop. Hence, the numbers are displayed using while and do-while loop.

13: Implementation of Constructors

Objective:

- To implement constructors

Source Code:

```
using System;
namespace Chapter_Three_Constructors
{
    internal class Program
    {
        public static int iRollNo;
        public static string myName;
        public int MyClass;
        static Program()
        {
```

```

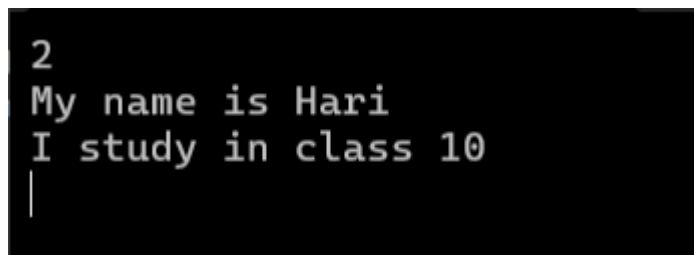
        iRollNo = 2;
        myName = "Hari";
    }
    public Program()
    {
        MyClass = 10;
    }
    static void Main(string[] args)
    {
        Program p = new Program();
        Console.WriteLine(iRollNo);
        Console.WriteLine("My name is " + myName);
        Console.WriteLine("I study in class " + p.MyClass);

        Console.ReadLine();

    }
}

```

Output:



```

2
My name is Hari
I study in class 10
|

```

Conclusion:

From this lab, we learned to implement static and non-static constructors. Hence, the constructors are created and the values are displayed.

14: Implementation of Inheritance

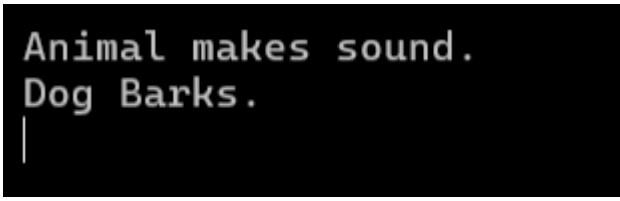
Objective:

- To implement inheritance

Source Code:

```
using System;
namespace Chapter_Three_Inheritance
{
    internal class Program
    {
        public class Animal
        {
            public void Sound(){
                Console.WriteLine("Animal makes sound.");
            }
        }
        public class Dog: Animal
        {
            public void MakeSound(){
                Console.WriteLine("Dog Barks.");
            }
        }
        static void Main(string[] args)
        {
            Dog dog = new Dog();
            dog.Sound();
            dog.MakeSound();
            Console.ReadLine();
        }
    }
}
```

Output:

A screenshot of a terminal window with a black background and white text. It displays the output of the program: "Animal makes sound." on the first line, "Dog Barks." on the second line, and a vertical cursor on the third line.

```
Animal makes sound.
Dog Barks.
|
```

15: Implementation of Abstract Class and Method

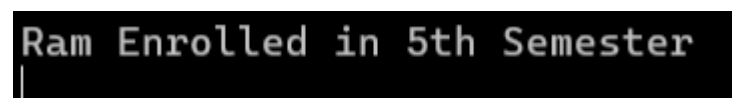
Objective:

- To implement abstract class and methods

Source Code:

```
using System;
namespace Chapter_Three_Abstract_Class
{
    internal class Program
    {
        public abstract class MyClass
        {
            public abstract void Enroll();
        }
        public class School : MyClass
        {
            public override void Enroll()
            {
                Console.WriteLine("Ram Enrolled in 5th
Semester");
            }
        }
        static void Main(string[] args)
        {
            School s = new School();
            s.Enroll();
            Console.ReadLine();
        }
    }
}
```

Output:



```
Ram Enrolled in 5th Semester
```

Conclusion:

From this lab, we learned to implement abstract class and method. Hence, an abstract class and method are created and displayed.

16: Implementation of Method Overloading

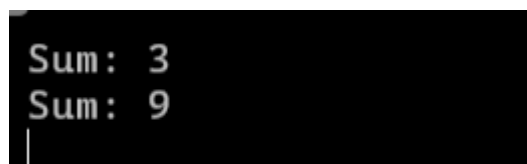
Objective:

- To implement method overloading

Source Code:

```
using System;
namespace Chapter_Three_Abstract_Class
{
    internal class Program
    {
        public class Calculation
        {
            public void Add(int x, int y)
            {
                Console.WriteLine("Sum: " + (x + y));
            }
            public void Add(int x, int y, int z)
            {
                Console.WriteLine("Sum: " + (x + y + z));
            }
        }
        static void Main(string[] args)
        {
            Calculation calc = new Calculation();
            calc.Add(1, 2);
            calc.Add(2, 3, 4);
            Console.ReadLine();
        }
    }
}
```

Output:



```
Sum: 3
Sum: 9
```

Conclusion:

From this lab, we learned to implement method overloading. Hence, two methods Add with different parameters are created and the results are displayed.

17: Implementation of Interface

Objective:

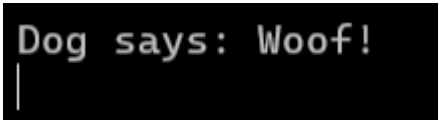
- To implement interface

Source Code:

```
using System;
namespace Chapter_Three_Interface
{
    internal class Program
    {
        interface IAnimal
        {
            void MakeSound();
        }
        class Dog : IAnimal
        {
            public void MakeSound()
            {
                Console.WriteLine("Dog says: Woof!");
            }
        }
        static void Main(string[] args)
        {
            IAnimal dog = new Dog();

            dog.MakeSound();
            Console.ReadLine();
        }
    }
}
```

Output:



Conclusion:

From this lab, we learned about interface. Hence, an interface Animal is created and it is implemented in class Dog and displayed.

18: Implementation of Generics

Objective:

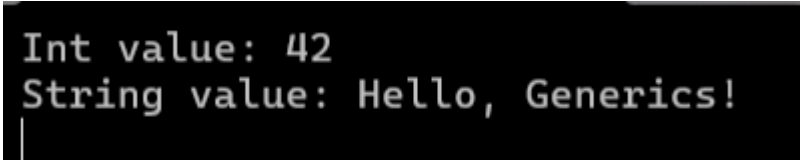
- To implement generics

Source Code:

```
using System;
namespace Chapter_Three_Generic
{
    internal class Program
    {
        class Box<T>
        {
            public T Value { get; set; }
        }
        static void Main(string[] args)
        {
            Box<int> intBox = new Box<int> { Value = 42 };
            Box<string> stringBox = new Box<string> { Value =
"Hello, Generics!" };

            Console.WriteLine($"Int value: {intBox.Value}");
            Console.WriteLine($"String value:
{stringBox.Value}");
            Console.ReadLine();
        }
    }
}
```

Output:

A screenshot of a terminal window with a black background and white text. It shows the output of the program: "Int value: 42" on the first line and "String value: Hello, Generics!" on the second line. A white cursor is visible at the end of the second line.

```
Int value: 42
String value: Hello, Generics!
```

Conclusion:

From this lab, we learned about generics. Hence, generics is created and the values are set and displayed.

19: Implementation of Delegates

Objective:

- To implement delegates

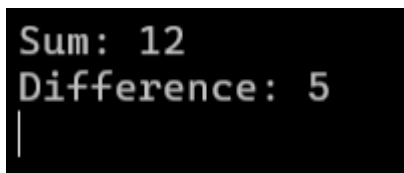
Source Code:

```
using System;
namespace Chapter_Four_Delegate
{
    internal class Program
    {
        public delegate void Calculation(int a, int b);
        static void Add(int a, int b) => Console.WriteLine("Sum: " + (a + b));
        static void Sub(int a, int b) => Console.WriteLine("Difference: " + (a - b));
        static void Main(string[] args)
        {
            Calculation c = Add;
            c(10, 2);

            c = Sub;
            c(10, 5);

            Console.ReadLine();
        }
    }
}
```

Output:



```
Sum: 12
Difference: 5
|
```

Conclusion:

From this lab, we learned to implement delegates. Hence, a delegate Calculation is created and methods are assigned and results are displayed.

20: Implementation of Lambda Expression

Objective:

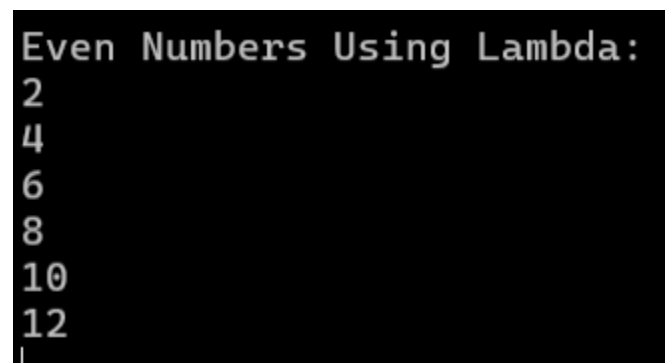
- To implement Lambda Expressions

Source Code:

```
using System;
using System.Collections.Generic;
namespace Chapter_Four_Lambda
{
    internal class Program
    {
        static void Main(string[] args)
        {
            List<int> mylist = new List<int> {
1,2,3,4,5,6,7,8,9,10,11,12,13};
            Console.WriteLine("\nEven Numbers Using Lambda:");
            List<int> listeven = mylist.FindAll(x => x%2 == 0);
            for (int i = 0; i < listeven.Count; i++)
                Console.WriteLine(listeven[i]);

            Console.ReadLine();
        }
    }
}
```

Output:



```
Even Numbers Using Lambda:
2
4
6
8
10
12
|
```

Conclusion:

From this lab, we learned to implement lambda expression. Hence, a list is created with numbers and even numbers are displayed using lambda expression.

21: Implementation of Exception Handling

Objective:

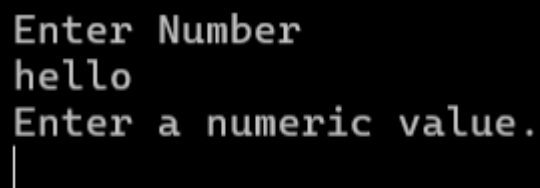
- To implement exception handling

Source Code:

```
using System;
namespace Chapter_Four_Try_Catch
{
    internal class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Enter Number");
            try
            {
                int num = Convert.ToInt32(Console.ReadLine());
                int iNo = num / 2;
            }
            catch (Exception ex) {
                if (ex.GetType().ToString() ==
                    "System.FormatException")
                {
                    Console.WriteLine("Enter a numeric value.");
                }
            }

            Console.ReadLine();
        }
    }
}
```

Output:



```
Enter Number
hello
Enter a numeric value.
|
```

22: Implementation of LINQ

Objective:

- To implement LINQ

Source Code:

```
using System;
using System.Collections.Generic;
using System.Linq;
namespace LINQ
{
    public class Student
    {
        public string Name { get; set; }
        public string Address { get; set; }
    }
    internal class Program
    {
        public static void Main(string[] args)
        {
            List<Student> students = new List<Student>
            {
                new Student { Name = "Ram", Address = "Kirtipur" },
                new Student { Name = "Hari", Address = "Kathmandu" },
                new Student { Name = "Sita", Address = "Kirtipur" },
            };
            var res = students.Where(s => s.Address ==
"Kirtipur").ToList();
            foreach (var student in res)
            {
                Console.WriteLine($"Name:{student.Name},Address:{
student.Address}");
            }
            Console.ReadLine();
        }
    }
}
```

Output:

```
Name:Ram,Address:Kirtipur
Name:Sita,Address:Kirtipur
```